

A new branching strategy for time constrained routing problems with application to backhauling

Sylvie Gélinas^a, Martin Desrochers^a, Jacques Desrosiers^b
and Marius M. Solomon^c

^a*GERAD and École Polytechnique de Montréal, Montréal, Canada H3C 3A7*

^b*GERAD and École des Hautes Études Commerciales de Montréal,
Montréal, Canada H3T 1V6*

^c*Northeastern University, Boston, MA 02115, USA*

In this paper, we explore a new branching strategy for branch-and-bound approaches based on column generation for the vehicle routing problems with time windows. This strategy involves branching on resource variables (time or capacity) rather than on network flow variables. We also examine criteria for selecting network nodes for branching. To test the effectiveness of the branching strategy, we conduct computational experiments on time window constrained vehicle routing problems where backhauling is permitted only after all the shipments to clients have been made. The branching method proved very effective. In cases where time was the more binding constraint, time-based branching succeeded in decreasing the number of nodes explored by two thirds and the total computation time by more than half when compared to flow-based branching. The computational results also show that the overall algorithm was successful in optimally solving problems with up to 100 customers. It produced an average cost decrease of almost 7% when backhauling was permitted as compared to the cost involved when the client and the distributor routes were distinct.

Keywords: Routing, scheduling, time windows, backhauling, column generation, branch-and-bound.

AMS(MOS) subject classification: 90B05, 90C06, 90C08, 90C10.

1. Introduction

For a great number of enterprises, the storing and delivery of merchandise are essential aspects of running their business. Often, the delivery aspect can be modeled as a vehicle routing problem with time windows (VRPTW). This problem consists of determining vehicle routes serving all clients of an enterprise exactly once within customer-specified time windows at minimum cost.

Over the last few years, optimization methods have shown promise for solving practical size VRPTW problems. Algorithms based on Dantzig–Wolfe decomposition/column generation schemes have emerged as very powerful optimal approaches (Desrosiers et al. [7]). Nevertheless, much research is still needed to expand the applicability of optimal algorithms to increasingly difficult problems. One area involves the branching strategies necessary to transform the linear solution into an integer one. The difficulty of combining integer programming methods and the Dantzig–Wolfe decomposition/column generation was pointed out as early as the first applications of the decomposition method to time constrained problems (Appelgren [1, 2]). Later, Chvátal [4] has also mentioned the difficulty of obtaining integer solutions for the classical column generation approach in the context of the Cutting-Stock Problem. To date, a single type of branching strategy compatible with the column generation approach has been proposed for the VRPTW (Desrochers et al. [6]).

The contribution of this paper consists of the development of a new type of branching strategy. We branch on resource variables (e.g., time or capacity) rather than branching on network flow variables. This branching strategy proved to be much more efficient than branching on flow variables. Furthermore, its applicability is far more general than the VRPTW. This strategy can be used for other problems involving resource variables such as vehicle routing and crew scheduling problems.

2. Solving the problem of vehicle routing with time windows

We model the VRPTW as a set-partitioning problem. This problem is solved by using a branch-and-bound method. At each node of the branching tree, we find the optimal solution to the linear relaxation by column generation. If this solution is integer and it provides a new and better upper bound, then it becomes the best solution found. If this solution is fractional, then one of two cases may arise. If the cost is less than the cost of the current best solution, then we further divide the problem, and descend to new branch nodes in search of an integer solution. Otherwise, we fathom the current branch node and we skip to another branch node that has a potentially attractive branching. Exploration of the branching tree terminates when all such branch nodes have been evaluated, at which point the best solution found is optimal.

To present the approach in detail, we first describe the underlying network. We denote by N the set of clients. We also use two nodes to represent the central depot: a source 0 for the departure of the vehicles from the depot, and a sink $n + 1$ for their return to the depot, where $|N| = n$. We define a network $G = (V, A)$, where $V = \{0, n + 1\} \cup N$ is the set of nodes considered and A is the set of admissible arcs. The set A is defined by $(\{0\} \times N) \cup I \cup (N \times \{n + 1\})$, where $I \subseteq N \times N$ is a subset described below.

In this network, each node possesses a time window $[a_i, b_i]$ and a capacity interval $[0, Q]$, where Q is the vehicle capacity. For the source and sink nodes, we

set $[a_0, b_0] = [0, 0]$ and $[a_{n+1}, b_{n+1}] = [0, T]$, where T is the maximum allowed duration of a single route. Each arc (i, j) has three associated values, t_{ij} , c_{ij} and q_{ij} . We define t_{ij} to be the sum of the time to service node i and the time to travel from node i to node j . We also define c_{ij} to be the distance between i and j . Finally, let q_{ij} be the load associated with the arc (i, j) . This is defined as the quantity of merchandise to deliver at node j , i.e., $q_{ij} = d_j$, $(i, j) \in A$ and $j \in N$, and $q_{i, n+1} = 0$, $\forall i \in N$. Now we can define the set I by eliminating from $N \times N$ arcs (i, j) that are not feasible with respect to time constraints, i.e., if $a_i + t_{ij} > b_j$, or with respect to capacity constraints, i.e., if $d_i + d_j > Q$.

Define now T_i to be the time at which service at node i begins and set $T_0 = 0$. If the arc (i, j) is used in the solution, we set the time variable $T_j = \max\{T_i + t_{ij}, a_j\}$, indicating that if the vehicle arrives early it must wait. We also define Q_i to be the load of the vehicle after visiting node i . Similarly to the recursive evaluation of the time variables, we have $Q_0 = 0$ and $Q_j = Q_i + q_{ij}$.

The graph G allows the construction of routes for the vehicles. A route r in G is feasible if it starts and finishes at the depot, and if it respects the time and capacity constraints of the visited nodes. The cost of the route r is calculated as being the sum of the costs of the arcs making up the route. Our objective is to find a set of feasible routes of minimum cost so that each client is visited exactly once.

2.1. THE SET-PARTITIONING MODEL

All time constrained vehicle routing problems can be formulated using the set partitioning model. This means that we must choose a set of feasible routes servicing each client $i \in N$ exactly once at minimum cost. Let R be the set of feasible routes, c_r the cost of the route $r \in R$, and a_{ir} a binary constant equal to 1 if node i is visited on route r and 0 otherwise, $i \in N$ and $r \in R$. Let Y_r be a binary variable corresponding to $r \in R$, equal to 1 if the route r is chosen, and 0 if not. Finally, let Y_v be the number of routes chosen, and Y_c the cost of the solution.

The problem is formulated as follows:

$$\text{Minimize } \sum_{r \in R} c_r Y_r \quad (1)$$

$$\text{subject to } \sum_{r \in R} a_{ir} Y_r = 1, \quad \forall i \in N, \quad (2)$$

$$\sum_{r \in R} Y_r - Y_v = 0, \quad (3)$$

$$\sum_{r \in R} c_r Y_r - Y_c = 0, \quad (4)$$

$$Y_r \in \{0, 1\}, \quad \forall r \in R, \quad (5)$$

$$Y_v, Y_c \geq 0 \text{ and integer.} \quad (6)$$

Constraints (1), (2) and (5) form a set-partitioning problem. The additional constraints (3), (4) and (6) which are redundant allow us to impose the integrality conditions on the number of vehicles used and on the cost of the solution. This model is solved by branch-and-bound where the lower bounds are computed by solving the linear relaxations corresponding to the respective nodes of the search tree. The branching procedure uses constraints (3), (4) and (6).

2.2. SOLVING THE LINEAR RELAXATION

By relaxing the integrality constraints (5) and (6) of the set-partitioning problem, we obtain a linear problem having a very large number of variables, each being associated with a feasible route of R . Hence, we will work with a reduced set R' of feasible routes where $R' \subset R$. Initially, the set R' could have n routes, one for each client to visit.

The simplex algorithm provides the optimal solution to the restricted linear problem defined on route set R' . It also provides the dual variables $\pi_i, i \in N, \pi_v$ and π_c . Since a variable (column) corresponds to a route, the solution found will be optimal for R if the marginal cost of all the routes $r \in R - R'$ have positive or zero marginal costs, i.e., if $\bar{c}_r \geq 0, \forall r \in R - R'$. By column generation, we obtain the route r^* whose marginal cost is the least amongst all the feasible routes, i.e., the route r^* such that $\bar{c}_{r^*} = \min_{r \in R} \bar{c}_r$. If $\bar{c}_{r^*} < 0$, route r^* is added to the set R' and the restricted linear problem is re-optimized. Otherwise, the optimization process is complete and the solution obtained for the set R' is then optimal for the set R . This solution provides a lower bound on the cost of the integer solution to problem (1)–(8).

Route r^* could be obtained by solving a subproblem defined as a shortest path problem with time and capacity constraints on the network $G = (V, A)$. However, this problem class is NP-hard in the strong sense (Dror [9]). Therefore, it is unlikely that even pseudo-polynomial algorithms could exist for its solution. Hence we consider a relaxed version which allows cycles but can be solved by a pseudo-polynomial dynamic programming algorithm. Details of this approach including those with respect to determining the service start times $T_i, i \in N$ and the loads $Q_i, i \in N$, are given in Desrochers et al. [6]. As pointed out by these authors, the set R can be enlarged to contain routes with cycles. In this case, a_{ir} represents the number of visits made to node i by route r . Note that this leads to a weakening of the lower bound.

If the optimal solution of the linear relaxation satisfies the integrality constraints, then this solution is also optimal for the set-partitioning problem. Otherwise, it is necessary to eliminate the solution from the feasible domain, either by adding cuts or by a branching technique.

2.3. BRANCHING ON FLOW VARIABLES

To obtain an integer solution, we first consider the variables Y_v and Y_c as presented in Desrochers et al. [6]. When these variables are integer, we only need

to branch so that the variables Y_i also become integer. It is here that the main difficulty mentioned in the introduction arises. It stems from the fact that fixing a fractional optimal basic variable at zero results in the regeneration of the corresponding column. This can be avoided if the subproblem is allowed to generate solutions up to the k th best one (see Hansen et al. [10], and Maculan et al. [13]). A simpler approach to obtain an optimal integer solution is to take all the decisions on variables appearing in the subproblem's formulation, i.e., the flow variables X_{ij} , $(i, j) \in A$, the time variables T_i , $i \in V$ or the load variables Q_i , $i \in V$.

The branching strategy proposed by Desrochers et al. [6] involves arcs belonging to the routes whose flow variables are fractional. The branching decisions can thus be easily transferred to the subproblem. A single arc is chosen using several criteria, more notably the value of the flow on the arc. Branching generates two problems. In the first problem, we impose the condition that the chosen arc be used and have a flow of 1. In the second problem, we impose the condition that the arc not be used, thereby having a flow of 0. When we impose the use of an arc (i, j) , we eliminate all arcs (i, k) with $k \neq j$ and (k, j) with $k \neq i$. Hence, after visiting node i , a vehicle must proceed immediately to node j . When we impose a zero flow on the chosen arc (i, j) , only the arc (i, j) is effectively removed from the network. Since a great number of arcs can be incident to node i or node j , the elimination of arc (i, j) only can have little effect on the solution, and the new problem may be almost as difficult as the one from which it was generated.

Therefore, we have developed a new branching strategy based on the division of the time intervals which are associated with the client nodes. Note that a similar strategy could instead involve the capacity intervals.

3. Branching on time variables

The linear relaxation of the set-partitioning formulation (1)–(6) characterizes the VRPTW in terms of flow variables on the arcs, and two cumulative resources, time and capacity, respectively. We propose branching on resource variables rather than branching on flow variables. This consists of splitting the time window with a node and creating a branch for each subinterval. Certain routes then become infeasible. Intuitively, this type of branching should be stronger than branching on flow variables since time and capacity constraints account for most of the difficulty of solving the VRPTW. In this paper, we only consider time since it is the most constraining resource in our test problems.

A similar strategy has already been used by Desrosiers et al. [8] in a network relaxation approach for the multiple traveling salesman problem with time windows. In this context, time constraints were discarded and the network relaxation was obtained. This produces an integer solution where certain routes will very likely be infeasible due to time windows not being satisfied. The branching divisions on the time windows were derived by selecting a certain infeasible route and splitting the

time windows on the nodes before the first infeasible one to remove this route. This strategy is no longer valid when a column generation approach is used. The linear relaxation of the master problem provides a fractional solution, i.e., several nodes are visited by more than a single route. To find the optimal integer solution, one has to split the time window at a chosen node so that only a single route serves that node. Hence, although both papers present branching strategies based on the division of the time windows, the two approaches are completely different.

We next characterize the time interval divisions which render the current fractional solution no longer feasible in the new problems. We then propose different strategies for choosing an effective division.

3.1. TIME WINDOW DIVISIONS

A node visited by a route with fractional flow will necessarily be visited by more than one route, or by the same route several times in a cycle. Suppose that we choose a network node i for branching. We divide the time window $[a_i, b_i]$ into two subintervals so that two new problems are created: one where $T_i \in [a_i, t]$, and the other where $T_i \in [t + 1, b_i]$, t being integer. The interval $(t, t + 1)$ is eliminated because the travel time and the service time are both integer. In the first problem, we impose the condition that node i be visited so that service starts no later than time t . We thus eliminate all routes which arrive at node i after time t . In the second problem, we impose the condition that node i be serviced no earlier than time $t + 1$. Vehicles arriving earlier than $t + 1$ will have to wait, and will therefore be delayed. These vehicles may therefore not have sufficient time to serve the clients who follow.

We devise a branching procedure so that the current solution becomes infeasible in each of the new problems. A node is a candidate node for branching if it has a time window that can be divided into two intervals, so that for each new problem generated, at least one route will no longer be feasible.

We designate by R^f the set of routes with fractional flow belonging to the current solution. To simplify the notation, we suppose first that a node is not visited more than once by a single route. We define the feasibility intervals $[l_i^r, u_i^r]$ for all nodes $i \in N$ and for all routes r of R^f such that $a_{ir} = 1$: l_i^r is the earliest time that service can start at node i on route r , and u_i^r is the latest time that service can start at node i on route r . The interval $[l_i^r, u_i^r]$ represents the time span during which node i must be visited in order for route r to remain feasible. Let $r \equiv (i_0, i_1, i_2, \dots, i_K)$, where $i_0 = 0$ and $i_K = n + 1$, and suppose r is a route with fractional flow belonging to the current solution. Then, the intervals are calculated recursively as follows:

$$l_{i_0}^r = 0; \quad l_{i_k}^r = \max\{l_{i_{k-1}}^r + t_{i_{k-1}, i_k}, a_{i_k}\}, \quad k = 1, \dots, K - 1; \quad (7)$$

$$u_{i_K}^r = T; \quad u_{i_k}^r = \min\{u_{i_{k+1}}^r - t_{i_k, i_{k+1}}, b_{i_k}\}, \quad k = K - 1, \dots, 1. \quad (8)$$

Note that when a route r visits the same node several times, i.e., $a_{ir} \geq 2$, $i \in N$ and $r \in R^f$, we have a feasibility interval for each of the visits. Then, it suffices to add an index indicating the interval associated with each visit.

If we find a node for which two visits, whether by the same route or two different routes, have disjoint feasibility intervals, then this node is a candidate for branching. To express this formally, we define

$$L_i = \max_{r \in R^f} \{l_i^r\}, \quad i \in N, \quad (9)$$

$$U_i = \min_{r \in R^f} \{u_i^r\}, \quad i \in N, \quad (10)$$

and derive the following result.

PROPOSITION 1

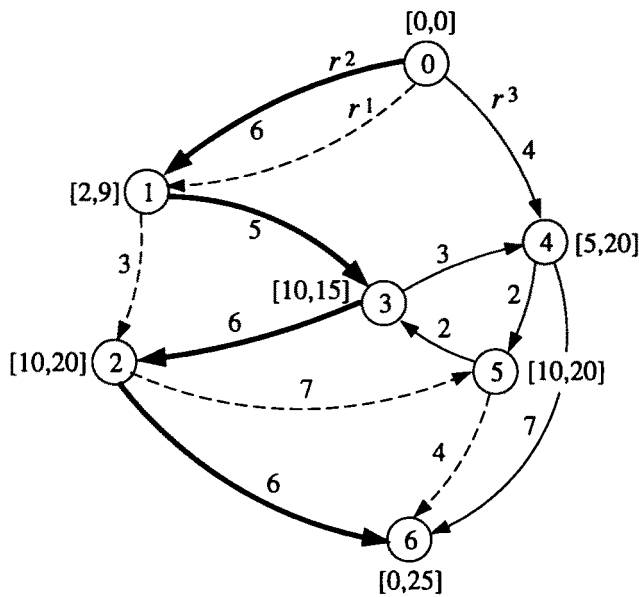
A node $i \in N$ is a candidate node for branching if and only if $L_i > U_i$.

Proof

Suppose first that $L_i > U_i$. Then there exists a route $r' \in R^f$ such that $L_i = l_i^{r'}$. This route is not valid if the time window of node i is restricted to the interval $[a_i, t_1]$ for $t_1 < L_i$. There also exists a route $r'' \in R^f$ such that $U_i = u_i^{r''}$. This route is not valid if the time window of node i is restricted to the interval $[t_2, b_i]$ for $t_2 > U_i$. By choosing t , such that $U_i \leq t < L_i$, we create a branching which results in one of the routes of the current solution not being feasible in each problem.

Suppose now that $L_i \leq U_i$. In a branching where two new problems are created, the resulting time windows at node i in each problem are of the form $[a_i, t]$ and $[t + 1, b_i]$, respectively, with $t \in [a_i, b_i]$ and integer. Consider a randomly chosen branching for a route $r \in R^f$. If $t \geq L_i$, the last visit to node i can be made at time l_i^r since $a_i \leq l_i^r \leq L_i \leq t$. The preceding visits to node i on route r will then occur before the time l_i^r . Hence, route r remains feasible for the problem involving the time window $[a_i, t]$. If $t < U_i$, the first visit to node i can be made at time u_i^r since $t + 1 \leq U_i \leq u_i^r \leq b_i$. The other visits to node i on route r will consequently be made after the time u_i^r . Therefore, route r remains feasible for the problem involving the time window $[t + 1, b_i]$. It follows that whatever the choice of partition, all the routes of the current solution remain feasible in at least one of the two new problems generated and the fractional solution is still optimal in the current problem. $\square$

The identification of candidate nodes is depicted in figure 1. In this figure, time windows appear in square brackets next to each node, and the travel time which includes the service time appears on each arc. For example, the time window for node 1 is $[2, 9]$ and the travel time on arc $(1, 2)$ is 3. We will assume that the capacity constraints are satisfied. There are three routes, $r^1 \equiv (0, 1, 2, 5, 6)$, $r^2 \equiv (0, 1, 3, 2, 6)$



Nodes	1	2	3	4	5
Routes	Intervals $[l_i^r, u_i^r]$				
r^1	[6, 9]	[10, 13]	–	–	[17, 20]
r^2	[6, 8]	[17, 19]	[11, 13]	–	–
r^3	–	–	[12, 15]	[15, 11]	[10, 13]
L_i	6	17	12	15	17
U_i	8	13	13	11	13
Candidates		◇		◇	◇

Figure 1. Identification of candidate nodes in a fractional solution.

and $r^3 \equiv (0, 4, 5, 3, 4, 6)$, each having 0.5 units of flow. The table below figure 1 shows the computation of the intervals $[l_i^r, u_i^r]$ and the values L_i and U_i for each node, $i = 1, \dots, 5$. Note that route r^3 contains a cycle passing twice through node 4. The first visit can be made in the interval $[5, 11]$ and the second in the interval $[15, 18]$. Then, nodes 2, 4 and 5 are candidates for branching, since $L_i > U_i$ for $i = 2, 4$ and 5. Therefore, a first branching could be made at node 2 where the split would be made at $t = 13 \in [13, 17)$. Route r^2 is not feasible in the interval $[10, 13]$, while route r^1 is not feasible in the interval $[14, 20]$. A second branching could be made at node 4, by dividing the original interval $[5, 20]$ at values $t \in [11, 15]$. In each of the branches created, route r^3 is not feasible. Other branchings could be made at node 5 with values $t \in [13, 17]$.

3.2. BRANCHING STRATEGIES

Let N_c be the set of candidate nodes derived from the proposition. If $N_c = \emptyset$, we could branch on either the flow or the load variables. We have opted for the former in light of the structure of the test problems. Suppose next that $N_c \neq \emptyset$. In order to eliminate the current solution, we branch by choosing a single node $i \in N_c$ and an integer value $t \in [U_i, L_i)$. We propose several strategies for choosing a network node on which to branch and we describe the rules used for dividing the time window of the chosen node.

3.2.1. Choosing a time window division

To generate a time window split, we consider all feasibility intervals, whether they be associated with the same or different routes. To simplify the notation, let us omit the index r identifying the route. Next, let Γ_i be the total number of visits made to node i by all fractional routes of R^f and also let $[l_i^\gamma, u_i^\gamma]$ be the feasibility interval for the γ th visit to node i in the current solution. In order for the route making this visit to remain feasible, the start of service must occur during this interval.

Suppose now that we have chosen node $i \in N_c$ for branching. We then must choose an integer value $t \in [U_i, L_i)$ in order to determine a division. This value is chosen by examining the flow passing through node i . The total value of this flow is equal to 1 and each visit made to this node contributes a certain amount of flow to this total. We divide the time window of node i in such a way that the total flow values eliminated from each side are as balanced and as large as possible.

For this, let $A_i(t)$ be the flow eliminated if we restrict the window of node i to the interval $[a_i, t]$ and let $B_i(t)$ be the flow eliminated if the restriction is the interval $[t + 1, b_i]$. To choose t , we determine the bounds l_i and u_i , $t \in [u_i, l_i)$, satisfying the following conditions:

$$[u_i, l_i) \subseteq [U_i, L_i) \text{ and } [u_i, l_i) \neq \emptyset, \quad (11)$$

$$l_i \in \{l_i^\gamma\}_{1 \leq \gamma \leq \Gamma_i} \text{ and } u_i \in \{u_i^\gamma\}_{1 \leq \gamma \leq \Gamma_i}, \quad (12)$$

$$l_i^\gamma \notin (u_i, l_i) \text{ and } u_i^\gamma \notin (u_i, l_i), \quad 1 \leq \gamma \leq \Gamma_i, \quad (13)$$

$$|A_i(u_i) - B_i(l_i)| \text{ is minimum.} \quad (14)$$

Condition (11) ensures that the current solution becomes infeasible in each of the new problems. Next, condition (12) states that the values l_i and u_i correspond to the boundaries of the feasibility interval. We also impose condition (13) for maximum elimination of flow, i.e., no feasibility interval starts or finishes within the interval bounded by u_i and l_i . Finally, condition (14) creates an equilibrated partition in terms of the flow eliminated in each branch.

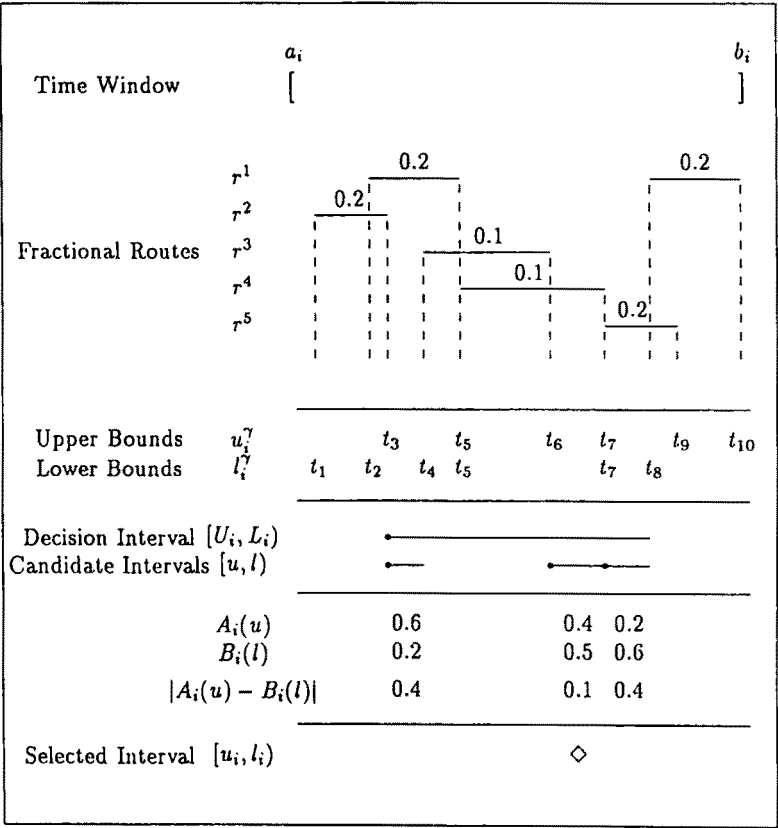


Figure 2. Determination of the interval $[u_i, l_i]$.

The choice of a time window division is highlighted in figure 2.

In this example, node i is served twice by route r^1 and once by routes $r^2, \dots, r^5$. We thus have six visits made to node i by the set of fractional routes, i.e., $\Gamma_i = 6$. Based on the flow values of the routes and their intervals of feasibility within $[a_i, b_i]$, the upper and lower bounds u_i^j and l_i^j of the feasibility intervals take only 10 different values, $t_1, t_2, \dots, t_{10}$. The decision interval $[U_i, L_i]$ is given by $[t_3, t_8]$. The three candidate intervals for the partition $[u_i, l_i]$ that respect the conditions (11), (12) and (13) are $[t_3, t_4]$, $[t_6, t_7]$ and $[t_7, t_8]$. The minimum quantity $|A_i(u_i) - B_i(l_i)|$ is achieved for $u_i = t_6$ and $l_i = t_7$, hence the time window split can be performed using any integer value $t \in [t_6, t_7]$.

3.2.2. Choice of the network node for branching

We propose three strategies for choosing the network node for branching. In the first strategy we branch so that important cycles will be eliminated. The other two strategies focus on the total number of visits made at each node, and on the flow values, respectively.

Elimination of cycles: The shortest path algorithm implemented by Desrochers et al. [6] eliminates the 2-cycles of the form (i, j, i) but does not eliminate the cycles of length greater than or equal to three. The existence of these cycles leads to appreciable decreases in the lower bound provided by the linear relaxation. We use branching on time variables in order to eliminate these important cycles and obtain better lower bounds. We first search for a route r containing a cycle whose flow is sufficiently high and then for the nodes visited more than once by this route. If we find a node i such that $u_i^r < l_i^r$, we choose this node for branching. By choosing the division of its time window in the interval $[u_i^r, l_i^r)$, we eliminate the cycle in each of the new problems, since one of the visits would no longer be allowed.

Number of visits: A node visited by a route with fractional flow will necessarily be visited by several routes or by one route several times. When we branch, some fractional routes may still remain feasible in one or both of the new problems. If these are numerous, it is likely that we would obtain fractional solutions once again. Intuitively, it seems preferable to choose a node visited only a few times over one visited many times in order to reduce the number of routes still feasible at this node in each of the new problems.

Flow values: The proposed procedure for dividing a time interval provides for each candidate node the quantity of flow which can be eliminated in each of the new problems. Our evaluation criterion depends on the total quantity of flow eliminated for the two problems, and on how close in value these quantities are to each other. Specifically, we choose a node $i \in N_c$ such that

$$i \in \arg \max_{k \in N_c} \{A_k(u_k) + B_k(l_k) - |A_k(u_k) - B_k(l_k)|\}. \quad (15)$$

Hence, this criterion involves choosing the candidate node for which the total quantity of flow eliminated for the two new problems is both maximal and best balanced between the two problems.

4. Computational experiments

To test the effectiveness of the proposed branch-and-bound strategy, we have conducted computational experiments in an important routing environment, the simple backhauling problem with time windows (VRPBTW). The rationale for choosing this environment is that, in the process of evaluating the branching schemes, it also permits to examine the potential benefits that could be derived from applying the overall optimization method to practical time window scenarios previously not examined in the literature. We constructed 45 sample problems based on the data provided by Solomon [16].

Three types of tests were conducted. The first involved a comparison among strategies for branching on time windows using different node selection criteria. The second was intended to determine whether there are advantages to branching on time intervals rather than branching on arcs. In the third, optimal solutions obtained when keeping the pick-up and delivery points on distinct routes were compared with optimal solutions obtained when backhauling was allowed.

4.1. THE ROUTING ENVIRONMENT

The VRPBTW is an extension of the VRPTW where both delivery points (clients or linehaul customers) and pick-up points (distributors or backhaul customers) are considered. We consider the simple backhaul variant where all deliveries must be made before backhauling is allowed. For reasons such as priority, access to merchandise, and security, all client linehaul shipments must be made before new goods are collected from backhaul clients. Most procedures which have been proposed to solve backhaul problems do not consider time windows. They are adaptations of insertion heuristics created for the classical vehicle routing problem (Deif and Bodin [5], Golden and Assad [12], Casco et al. [3], Goetschalckx and Jacobs-Blecha [11]). To our knowledge, only one mathematical optimization method has been used with success for a chain of 40 stores (Yano et al. [17]). This method is based on a set-covering formulation and on the a priori enumeration of a subset of admissible routes.

For the VRPBTW, the underlying network introduced in section 2 requires several modifications. We denote by N_d the set of linehaul clients (for deliveries), by N_p the set of backhaul clients (for pick-ups), and by N the union of the two. For each node $i \in N$, we have the quantity d_i to deliver if i is a client in N_d , or the quantity p_i to pick-up if i is a distributor in N_p . In the network $G = (V, A)$, the set of admissible arcs is defined by $(\{0\} \times N) \cup I \cup (N \times \{n+1\})$, where $I \subseteq (N_d \times N_d) \cup (N_d \times N_p) \cup (N_p \times N_p)$. Note that by omitting the distributor-client arcs ($N_p \times N_d$), we are assured that the distributors will be visited only after all the clients have been serviced. Note also that we can further reduce the set A by eliminating the arcs (i, j) such that $p_i + p_j > Q$.

We define by Q_i the load of the vehicle after visiting node i , if $i \in N_d$, or before visiting node i , if $i \in N_p$, and we set $Q_0 = 0$. Depending on the pair (i, j) , the arc load variable q_{ij} is defined as: $q_{0,j} = 0$, for $j \in N_p$ and $q_{i,n+1} = 0$, for $i \in N_d$. For each client $j \in N_d$, the quantity of merchandise to deliver, d_j , is placed on the arcs entering j , i.e., we set $q_{ij} = d_j$ for $i \in N_d \cup \{0\}$, and $j \in N_d$. For each distributor $i \in N_p$, the quantity of merchandise to pick-up, p_i , is placed on the arcs leaving i , i.e., we set $q_{ij} = p_i$ for $i \in N_p$, and $j \in N_p \cup \{n+1\}$. Then if the arc (i, j) is used in the solution, $Q_j = Q_i + q_{ij}$. In order to indicate that when all the deliveries have been completed a vehicle's full capacity is available for backhauling, we set the vehicle load to zero. This is accomplished by assigning $q_{ij} = -Q$ for the arcs $(i, j) \in N_d \times N_p$. Thus, if the arc $(i, j) \in N_d \times N_p$ is used in the solution,

$$Q_j = \max\{Q_i + q_{ij}, 0\} = \max\{Q_i - Q, 0\} = 0, \quad \forall j \in N_p.$$

Note that the capacity resource can be modeled in different but equivalent ways. Alternatively to the above model, we could instead define capacity windows $[0, 0]$ for the node 0, $[0, Q]$ for the nodes in N_d , and $[Q, 2Q]$ for the nodes in $N_p \cup \{n + 1\}$. This alternate model corresponds to imagining the vehicles as having twice their real capacity, with pick-ups being treated as a second set of deliveries.

4.2. DESCRIPTION OF THE TEST PROBLEMS

To create test problems for the VRPBTW, we used the first 5 problems of the $r1$ series developed by Solomon [16] for vehicle routing with time windows, the distance data being truncated to one decimal place. Each problem consists of 100 clients to be serviced during a short time horizon. The vehicle capacity constraints are fairly loose when compared to the time window constraints.

We constructed our test problems by randomly choosing 10%, 30% and 50% of the 100 clients in Solomon's problems to be backhaul clients instead. At these nodes, deliveries became pick-ups with the quantity of handled merchandise unchanged. We then generated additional test problems by considering only the first 25 and first 50 nodes, be they linehaul or backhaul clients. We thus obtained a total of 45 test problems for the VRPBTW.

The algorithm was programmed in FORTRAN77 and was integrated within the GENCOL software package (Sans   et al. [15]). The version we used incorporates the linear program XMP of Marsten [14]. Our tests were executed on a SUN Sparc station 1 (12.5 mips, 1.2 Mflops).

4.3. EXPERIMENTS ON BRANCHING STRATEGIES

The objective of these tests was twofold. First, to evaluate the effectiveness of the different node selection criteria proposed for branching on resource variables. Second, to examine the potential benefits of resource-based branching by comparing it with flow-based branching. Since the test problems considered had time as the constraining resource, it was this resource that we used for branching. We limited the exploration of the branching tree to a maximum of 200 branch nodes explored using a depth-first policy.

4.3.1. Node selection for branching on time variables

To determine the best criterion for selecting a node for branching on time variables, we picked seven test problems of medium difficulty. We judged difficulty in terms of the number of nodes explored when solving the problem using branching on flow variables. For each problem we conducted several runs which differed in terms of the criteria used to choose the network node for branching. The results are given in table 1.

Table 1

Number of branch nodes explored for the various node selection criteria.

Problem	Identification		Selection criteria for the candidate node				
	Nodes	% distributors	FLOW	VISIT ⁺	VISIT ⁻	FLOW/VISIT ⁻	CYCLES/FLOW/VISIT ⁻
r102	100	50	18	200*	40	6	6
r103	50	10	8	126	62	12	12
r103	50	30	16	68	22	16	14
r103	50	50	4	10	8	4	4
r104	25	10	28	200*	76	40	18
r105	25	30	24	56	26	24	24
r105	50	10	34	200*	56	28	28

*Optimality has not been reached within the limit of 200 branch nodes explored.

As presented in this table, the first criterion was that associated with the flow values, as described in section 3.2 and denoted by FLOW in table 1. We also examined two criteria where the choice of node was dependent on the number of visits. In one of these tests, nodes visited many times were preferred. In the other, nodes visited only a few times were preferred. These criteria are denoted by VISIT⁺ and VISIT⁻ in table 1.

As can be seen from the table when comparing the above three criteria, FLOW was the most effective. It resulted in the least number of branch nodes explored before an optimal solution was found. The results support the hypothesis that it is more effective to select a node visited only a few times rather than one visited many times.

Based on these results, the two criteria associated with the flow values and the number of visits to a node were next combined in the type of branching depicted by FLOW/VISIT⁻ in table 1. This method first chooses a node according to its flow values. If there are several best candidates, it then selects the one which has the least number of visits. FLOW/VISIT⁻ turned out to be an equally effective criterion.

Next, we tested the criterion concerning the elimination of cycles. It requires a parameter α indicating the minimum value of flow necessary for a route to be considered. We tested values of α equal to 0.15, 0.2, 0.25, 0.3 and 0.35 on several test problems. When α equaled 0.3 or 0.35, we obtained results at least as good as those obtained by the aforementioned criteria. However, when $\alpha = 0.15$, the strategy of eliminating cycles was not very effective. This is because a route with flow as small as 0.15 could visit a node up to six times. So even if the first cycle was eliminated, the second through the sixth would sometimes still remain feasible.

Hence, to account for the importance of eliminating cycles and to take advantage of the quality of the FLOW/VISIT⁻ criterion, we used them together in a combination criterion denoted by CYCLES/FLOW/VISIT⁻ in table 1. A node is chosen using first

the cycle elimination criterion, CYCLES with $\alpha = 0.3$. When no such node exists, the second criterion, FLOW/VISIT⁻ is used to select the node. As illustrated in table 1, this combination produced results better or equal to those obtained by FLOW/VISIT⁻ alone.

4.3.2. Branching on time versus flow variables

Another set of tests was conducted to compare branching on time variables with branching on flow variables. In the former method, we used the CYCLES/FLOW/VISIT⁻ criterion. The results for the 21 out of 45 problems requiring branching are given in table 2.

Table 2
Branching on flow variables versus time variables.

Identification			Branching strategy			
Problem	Nodes	% distributors	Flow variables		Time variables	
			B&B nodes explored	CPU time (sec) ^a	B&B nodes explored	CPU time (sec) ^a
r101	50	10	4	38	4	34
		100	16	781	4	264
		30	90	1608	2	162
		50	30	770	4	166
r102	25	10	2	15	2	12
		50	6	20	4	15
	100	10	12	3191	4	1988
		30	2	969	6	1362
		50	54	3606	6	860
r103	25	30	10	86	10	54
		10	8	771	12	813
		30	44	1051	14	397
		50	10	346	4	225
r104	25	10	48	1340	18	426
	50	50	176	11702	200*	13679
r105	25	10	6	16	2	11
		30	62	80	24	26
		50	2	9	8	14
	50	10	106	991	28	324
		30	24	191	16	128
		50	12	118	14	105

^a Total competition time on a SUN Sparc station 1 (12.5 mips, 1.2 Mflops).

* Optimality has not been reached within the limit of 200 nodes explored.

Table 3
Computational results on backhauling.

Identification			2-VRPTW versus VRPBTW			Integrality gap	
Problem	Nodes	% distributors	IP _{2-VRPTW}	IP _{VRPBTW}	% savings	LP _{VRPBTW}	% gap
r101	25	10	670.4	643.4	4.0	643.4	0
		30	775.5	711.1	8.3	711.1	0
		50	764.5	674.5	11.8	674.5	0
	50	10	1168.6	1122.3	4.0	1121.7	0.05
		30	1267.2	1191.5	6.0	1191.5	0
		50	1262.7	1168.6	7.5	1168.6	0
	100	10	1823.4	1767.9	3.0	1765.9	0.11
		30	1915.4	1877.6	2.0	1874.4	0.17
		50	1951.8	1895.1	2.9	1894.1	0.05
r102	25	10	575.0	563.5	2.0	563.3	0.04
		30	664.5	622.3	6.4	622.3	0
		50	670.4	584.4	12.8	583.4	0.17
	50	10	1042.1	974.7	6.5	974.7	0
		30	1075.5	1024.8	4.7	1024.8	0
		50	1144.1	1057.2	7.6	1057.2	0
	100	10	1673.3	1600.5	4.4	1595.9	0.29
		30	1705.5	1639.2	3.9	1639.0	0.01
		50	1773.2	1721.3	2.9	1712.5	0.51
r103	25	10	495.3	476.6	3.8	476.6	0
		30	560.9	507.0	9.6	500.4	1.30
		50	603.3	475.6	21.2	475.6	0
	50	10	900.7	811.4	9.9	808.1	0.41
		30	954.7	882.8	7.5	880.6	0.25
		50	969.7	882.1	9.0	881.9	0.02
r104	25	10	472.4	452.5	4.2	444.1	1.86
		30	541.2	467.6	13.6	467.6	0
		50	564.2	446.8	20.8	446.8	0
	50	50	787.5	733.6	6.8	726.0	1.04
r105	25	10	572.7	565.1	1.3	563.8	0.23
		30	649.3	623.5	4.0	611.8	1.88
		50	621.7	591.1	4.9	589.6	0.25
	50	10	1018.3	970.6	4.7	966.3	0.44
		30	1035.0	1007.5	2.7	1005.8	0.17
		50	1060.4	993.4	6.3	989.4	0.40

Branching on time variables proved clearly superior to branching on flow variables for the problems $r101$ with 100 clients. As these problems are characterized by small time windows, their division had a significant effect on fractional solutions. However, this type of branching did not succeed in solving problem $r104$ having 25 clients and 25 distributors. As this problem has large time windows, even if we eliminate some routes by branching, the new intervals are large enough that numerous other routes of similar cost may still be feasible. The presence of cycles in the linear relaxation solution is a major difficulty since many divisions are necessary in order to eliminate them. For this problem, branching on arcs proved to be the better method since it found an optimal integer solution. Overall, in all test cases except this one, branching on time intervals was more effective than branching on arcs, producing a 66% reduction in the number of branching nodes explored and a 54% reduction in total execution time.

4.4. COMPUTATIONAL RESULTS ON BACKHAULING

To determine the importance of backhauling and of applying optimization methods to such problems, we have compared for each problem the optimal solution obtained when clients and distributors are served on separate routes with the optimal solution derived when backhauling is permitted. Hence, we have solved two VRPTW, one for clients only, and the other for distributors only. We denote the sum of the costs of the two solutions by $IP_{2-VRPTW}$. We then solved the VRPTW for clients and distributors together, allowing backhauling. The cost of a solution to the backhauling problem is represented by IP_{VRPTW} . The computational results are presented in table 3. The algorithm was successful in solving to optimality realistic size problems. Overall, 34 out of the 45 problems tested were optimally solved. The largest problem solved involved 100 customers. For these problems, there was an average decrease of 6.8% in the total travel costs when backhauling was allowed as compared to where it was not an option. Therefore, given the substantial cost involved in routing and scheduling of vehicles, backhauling is a very attractive alternative which can provide substantial savings. These savings are not only illustrative of the economic attractiveness of backhauling but also of the optimization algorithms capable of solving such problems.

We have also measured the quality of the linear relaxation for the 34 test problems that were solved to optimality. In 13 cases (38%), the linear relaxation of the set-partitioning problem directly resulted in an integer solution. In the other 21 cases, the integrality gap averaged only 0.46%. Some difficult problems were never completely solved. These were $r103$, $r104$, $r105$ with 100 nodes, and $r104$ with 50 nodes of which 10% and then 30% were distributors.

5. Conclusions

In this paper we have explored a new branching strategy for branch-and-bound approaches based on column generation for the VRPTW. This strategy of branching

on resource variables proved much more effective than branching on flow variables. In particular, when time was the resource chosen, a network node selection criterion based on a combination of cycle elimination, flow value and the least number of visits to a node produced superior results. This new type of branching has applicability to any type of VRPTW. In applications where capacity is the more binding resource, one could branch on capacity rather than time intervals. In general, the more constraining a resource, the more effective it will be when used in branching. This strategy can therefore be effectively utilized for other discrete optimization problems involving resource variables.

Our results and earlier ones obtained by Desrosiers et al. [8] highlight the importance of selecting an appropriate type of variable to branch on. For the VRPTW, time and load variables have a more global nature than flow variables. For example, an n -client problem will only have n time or load variables compared to n^2 flow variables. This branching strategy is useful even when the integrality gap is large and the optimization approach is turned into a heuristic. It should be pointed out that while it represents a step in the right direction, branching is not sufficient to optimally solve VRPTW problems when large integrality gaps are present. For example, problems in the rc set of Solomon [16] will necessitate in addition to effective branching strategies much more powerful tools to bridge the integrality gap.

Acknowledgements

This research was supported by the Quebec Government (Fonds pour la Formation de Chercheurs et l'Aide à la Recherche) and by the Natural Sciences and Engineering Research Council of Canada. Marius M. Solomon was partially supported by the Patrick F. and Helen C. Walsh Research Professorship.

References

- [1] L.H. Appelgren, A column generation algorithm for a ship scheduling problem, *Transportation Science* 3(1969)53–58.
- [2] L.H. Appelgren, Integer programming methods for a vessel scheduling problem, *Transportation Science* 5(1971)64–78.
- [3] D. Casco, B. Golden and E. Wasil, Vehicle routing with backhauls: models, algorithms, and case studies, in: *Vehicle Routing: Methods and Studies*, ed. B.L. Golden and A.A. Assad (Elsevier Science, 1988) pp.127–147.
- [4] V. Chvátal, *Linear Programming* (Freeman, 1983).
- [5] I. Deif and L. Bodin, Extension of the Clarke and Wright algorithm for solving the vehicle routing problem with backhauling, *Conference on Computer Software Uses in Transportation and Logistics Management*, ed. A.E. Kidder, Babson Park, MA (1984) pp.75–96.
- [6] M. Desrochers, J. Desrosiers and M. Solomon, A new optimization algorithm for the vehicle routing problem with time windows, *Operations Research* 40(1992)342–354.
- [7] J. Desrosiers, Y. Dumas, M. Solomon and F. Soumis, Time constrained routing and scheduling, GERAD, École des Hautes Études Commerciales, Montréal, Canada (1992), to appear in *Handbooks in Operations Research and Management Science* (North-Holland).

- [8] J. Desrosiers, F. Soumis, M. Desrochers and M. Sauvé, Routing and scheduling with time windows solved by network relaxation and branch-and-bound on time variables, *Computer Scheduling of Public Transport 2*, ed. J.-M. Rousseau (Elsevier Science, 1985) pp.451–469.
- [9] M. Dror, Note on the complexity of the shortest path models for column generation in VRPTW, *Operations Research* 42(1994)977–978.
- [10] P. Hansen, B. Jaumard and P. Poggi de Aragão, Un algorithme primal de programmation linéaire généralisée pour les programmes mixtes, *Comptes Rendus de l'Académie des Sciences de Paris* 313(1991)557–560.
- [11] M. Goetschalckx and C. Jacobs-Blecha, The vehicle routing problem with backhauls, *European Journal of Operational Research* 42(1989)39–51.
- [12] B.L. Golden and A.A. Assad, The simple backhaul problem, Internal memo, Distribution Systems Technologies Inc., Columbia, MD (1984).
- [13] N. Maculan, P. Michelon and G. Plateau, Column-generation in linear programming with bounding variable constraints and its application in integer programming, Working Paper ES-268/92, Federal University of Rio de Janeiro, Brazil (1992).
- [14] R.E. Marsten, The design of the XMP linear programming library, *ACM Transaction on Mathematical Software* 7(1981)481–497.
- [15] B. Sansó, M. Desrochers, J. Desrosiers, Y. Dumas and F. Soumis, Modeling and solving routing and scheduling problems: GENCOL User Guide, Preliminary Version, GERAD, École des Hautes Études Commerciales, Montréal, Canada (1990).
- [16] M. Solomon, Algorithms for the vehicle routing and scheduling problem with time window constraints, *Operations Research* 35(1987)254–265.
- [17] C. Yano, T. Chan, L. Richter, T. Cutler, K. Murty and D. McGettigan, Vehicle routing at quality stores, *Interfaces* 17(1987)52–63.